

2. Analyze the two GenAI programs. The analysis must include:

- a. Name of two GenAI programs
 - i. Google Gemini flash 3.8
 - ii. Claude-Sonnet 4.6
- b. How you accessed them (e.g., Google query, CoPilot on my computer)
 - i. I used antigravity (Google's AI harness) and accessed both models that way.
- c. Prompt used. (You must use the same prompt for both GenAIs.)
 - i. I used this image as my prompt

f. Use two GenAI's to generate C code to implement the following requirements:

Objective

Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use `printf` to prompt the user and `scanf` to read their guess.
- Use a `while` or `for` loop to let the user guess up to 3 times.
- Use `if-else` statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

```
text
Guess a number between 1 and 10.
Attempt 1/3. Enter your guess: 5
Too low! Try again.
Attempt 2/3. Enter your guess: 8
Too high! Try again.
Attempt 3/3. Enter your guess: 7
Correct! You win!
```

ii.

d. Generated code from both GenAIs.

```
#include <stdio.h>

int main(void) {
    int secret = 7;
    int guess;
    int attempts = 0;

    printf("Guess a number between 1 and 10.\n");

    while (attempts < 3) {
        attempts++;
        printf("Attempt %d/3. Enter your guess: ", attempts);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            return 0;
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    printf("Sorry, you lose! The number was %d.\n", secret);
    return 0;
}
```

i. Claude:

```

#include <stdio.h>

int main(void) {
    // Secret number fixed between 1 and 10
    const int secret_number = 7;
    const int max_attempts = 3;
    int guess;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    // Loop to allow the user to guess up to 3 times
    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt, max_attempts);
        scanf("%d", &guess);

        // Check guess against the secret number
        if (guess == secret_number) {
            won = 1;
            break; // Stop the loop immediately if the user guesses correctly
        } else if (guess < secret_number) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    // Print final win or lose message
    if (won) {
        printf("Correct! You win!\n");
    } else {
        printf(
            "Sorry, you ran out of attempts. You lose! The secret number was %d.\n",
            secret_number);
    }

    return 0;
}

```

- ii. Gemini: }
 - e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
 - i. Correctness
 1. Gemini: the program runs, but it has no handling of invalid input. When you enter a string/character, the program doesn’t error out, but it does instantly use up all your guesses and then run the “sorry, you ran out of attempts...” message. It essentially silently crashes by just wrapping itself up; it has some weird error behaviour. It also has nothing for if you enter an input > 10 or < 1, and it just says either too high or too low like it would for any other number.
 2. Claude: the program runs as well, and it has the same issues as the gemini program. Neither have safeguards against characters/string inputs. Technically, I guess, a char input IS an int when stored in an int variable, but both programs instantly wrap up and use all guesses left in some weird error behaviour. Again, this program also has nothing for if the int is outside the 1-10 range.
 3. Extra bonus thing! Both programs have a minor correctness issue where after your final guess, if you get it wrong, it will still say a message

including "Try again.". Since you have used your last guess, it shouldn't say this in terms of correctness, but both do.

ii. Execution Time / Time complexity

1. Both programs are $O(1)$ constant time complexity, since both loops are only 3 loops long. However, there are minor things that make Claude potentially faster. Firstly, the similarities:

- a. Both programs have loops that run 3 times, and according to google, these two loops compile to basically the same machine code, so the choice of loop makes no difference.
- b. The logic in both programs' main code is essentially the same, with even the same error messaging (except for the loss message).

Claude's code is faster because of two minor things:

- a. When the player wins, it instantly prints the win message returns 0 and ends the program for Claude's code. Gemini has a "won" flag, sets that to 1, breaks, and then the program must fall until it reaches the end logic.
- b. In Claude's program, line 12

```
(printf("Attempt %d/3. Enter your guess: ",  
attempts);)
```

only calls %d once to have a variable put into the output, but Gemini's version, line 14:

```
(printf("Attempt %d/%d. Enter your guess: ",  
attempt, max_attempts);)
```

calls %d twice with two separate variables. This is slower.

iii. Space Time / Space Complexity

1. The space complexity is heavily (in regards to what can be maximized out of this tiny program) in favour of Claude's program. Claude only has 3 variables, Gemini has 4. According to my AI analysis, Claude's local storage is only 12 bytes, but Gemini is 20 bytes. Beyond that, both programs use the same amount of allocated space, but it may be argued that Gemini's space complexity is helped by the for-loop, but nothing I can find supports this and makes it more important that the extra variable.

iv. Maintainability

1. Finally, Gemini can get a win. Claude left exactly ZERO comments, while Gemini left a few good comments, not enough on certain places, but it is infinitely better than the lack of comments on Claude's. Claude is hurt by its efficiency here as well, since it uses a magic number (the 3 attempts) as opposed to Gemini's clearly marked "max_attempts" constant.

f. Select one to use as the basis for this assignment and justify why you picked it.

- i. I am picking Claude's code. I want to add comments and make the code actually legible because the code is much better than Gemini's but the readability is dogwater.
- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
 - i. Correctness:
 - 1. I improved claude.c's correctness by adding input handling and by making some unchanging variables like the secret number a constant. I also made it so that it won't say "Try again!" when you just used your last guess. I also added "in x attempts" to the welcome line so that the user knows they have limited attempts from just that line alone.
 - ii. Execution Time and Spatial Complexity:
 - 1. I did not improve the execution time nor the spatial complexity of the program, and if anything I made it worse, but it was for a good cause! In terms of execution time, I slowed the program down by adding multiple new logic blocks that have to run, and I also added the double %d to the "attempts left" line. Spatial complexity also took a hit, since I added an extra variable, which will take up more bytes, and I added more logic, which means more machine code. I had to do this, because when it comes down to it I'd rather have a properly functioning program that is "feature" complete than have a fast one that breaks when the user enters anything other than an integer. If you disagree, I'm sorry!
 - iii. Maintainability:
 - 1. I added every single comment that appears in the program, so I heavily increased the maintainability. Beforehand, you better know how to program C, or else you had no chance! I also removed the magic number 3 so that the number of attempts can fluctuate via a single line of code changing. I clearly labeled everything, and made it more modular, but at the expense of like maybe, MAYBE 1ms of runtime. I think it was worth.